

The CLI and CI/CD automation

Authentication without a human, configuration profiles, and the shape of an automated deployment pipeline.

In CI/CD there is **nobody there to log in.**
Every design decision in this module
follows from that one constraint.

What the CLI is

- ◆ **A command-line client for the Databricks REST APIs**, covering both workspace and account operations.
- ◆ **It is how bundles are deployed.** Every bundle command from module 21 is a CLI command.
- ◆ **It runs anywhere:** your laptop, a CI runner, a container.
- ◆ **Anything the CLI can do, a pipeline can do**, which is the point for this domain.

ON THE EXAM

The exam guide names the CLI for validating, deploying and managing bundles and other workspace assets. That is its role here.

Command groups you should recognise

GROUP	WHAT IT MANAGES
<code>databricks bundle</code>	Bundles: init, validate, deploy, run, summary, destroy
<code>databricks jobs</code>	Jobs: list, get, run-now, cancel
<code>databricks clusters</code>	Compute: create, start, list
<code>databricks secrets</code>	Secret scopes and secrets
<code>databricks fs</code>	Files in cloud storage and volumes
<code>databricks workspace</code>	Notebooks and workspace objects: import, export
<code>databricks auth</code>	Login and profile management

ON THE EXAM

The pattern is `databricks <group> <command>`. If you can name the group, you can usually guess the command.

Configuration profiles

- ◆ **Profiles live in a `.databrickscfg` file**, each with a name in square brackets and its own settings.
- ◆ **Select one with `--profile` or `-p` on any command.**
- ◆ **A workspace profile holds a host and credentials.** An account profile also holds an `account_id`.
- ◆ **Profiles are how one machine talks to several workspaces** — dev, staging and prod side by side.

ON THE EXAM

If a bundle target specifies a host and no token is set, the CLI looks for a profile in `.databrickscfg` with a matching host.

U2M or M2M?

OAuth U2M

User to machine

- You log in interactively through a browser
- The CLI manages short-lived tokens for you
- Tokens expire in under an hour
- For a person at a keyboard

OAuth M2M

Machine to machine

- A service principal and a client credential flow
- No interactive sign-in at any point
- Configured with client_id and client_secret
- For scripts, services and CI/CD

THE TIEBREAKER

The names say it. If a person is present, U2M. If nobody is, M2M. A CI runner is nobody.

Personal access tokens are the legacy option

- ◆ Databricks documentation labels PAT authentication legacy.
- ◆ OAuth M2M is the current recommendation for unattended access.
- ◆ A PAT is long-lived, so an exposed one stays valid until somebody notices and revokes it.
- ◆ OAuth tokens are short-lived and requested on demand, so exposure has a much smaller window.

ON THE EXAM

Older CI examples all use a PAT in a secret variable. It still works. It is no longer what Databricks points you at.

Credentials in a pipeline

- ◆ **Set them as environment variables** from your CI system's secret store.
- ◆ **Environment variables take precedence over .databrickscfg** for OAuth M2M, which is what makes this work.
- ◆ **Never commit a credential to the repository.** Module 20 covered why that is permanent.
- ◆ **Give the CI service principal only the permissions it needs** in each target workspace.

ON THE EXAM

A CI runner has no .databrickscfg. Environment variables are how the credential reaches a container that was created 30 seconds ago.

The shape of a deployment pipeline

01

On pull request

Validate the bundle. Fail the check if it does not resolve.

02

Run tests

Deploy to a dev target and run the job, or run unit tests.

03

Merge

Review and approve. The branch protection is the gate.

04

Deploy to staging

From the main branch, to the staging target.

05

Deploy to production

From main, as a service principal, in production mode.

WHERE THE JUDGEMENT LIVES

Notice where the human is. Reviewing a pull request, not running a deployment command.

Why validate belongs in CI

- ◆ **It checks the configuration resolves** without touching any workspace.
- ◆ **It catches a mistyped variable, a missing target, malformed YAML** — before anything is deployed.
- ◆ **It is fast and free**, which is why it belongs on every pull request.
- ◆ **It cannot catch a logic error in your notebook.** It validates configuration, not correctness.

ON THE EXAM

The exam objective lists validate before deploy and manage. That ordering is not accidental.

Deploying from the pipeline

- ◆ **Always name the target explicitly** with `-t`. Omitting it uses the default, which is how the wrong workspace gets deployed to.
- ◆ **Production mode validates the Git branch**, so a production deploy from a feature branch fails.
- ◆ **Set `run_as` to a service principal** so production assets do not belong to a person.
- ◆ **Avoid `--force` in a pipeline**. It exists to override validations, which is the opposite of what automation wants.

ON THE EXAM

A pipeline that routinely passes `--force` has turned its safety checks into decoration.

What to test, and where

- ◆ **Unit tests on your transformation logic** run without any workspace at all.
- ◆ **Integration tests deploy to a dev target and run the job**, then check the output.
- ◆ **bundle destroy** tears the test environment down afterwards, which keeps the cost bounded.
- ◆ **Data quality checks from module 14 run in the pipeline too**, on real data rather than on code.

ON THE EXAM

Deploy, run, assert, destroy is a complete integration test, and it is four CLI commands.

Managing other assets

- ◆ **The objective says bundles and other workspace assets.** The CLI covers both.
- ◆ **Secret scopes can be created and populated** from a pipeline, so no human ever sees the value.
- ◆ **Connections for Lakeflow Connect can be created programmatically**, except where browser OAuth is the only auth.
- ◆ **Workspace objects can be imported and exported**, which is how one-off migrations get scripted.

ON THE EXAM

Module 06's exception applies here: connectors whose only authentication is browser-based OAuth cannot be created programmatically.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
A pipeline must authenticate with no human present	OAuth M2M with a service principal
A developer logging in from their laptop	OAuth U2M, via databricks auth login
An older example using a token in a secret	Works, but PATs are the legacy option now
It works locally but fails in CI	The runner has no profile. Set environment variables.
A broken bundle reached production	No validate step on the pull request
Deployed to the wrong workspace	The target was not named with -t
The pipeline passes --force every time	The safety checks have become decoration

ON THE EXAM

Seven lines. The fourth is the single most common practical failure when a team first automates a deployment.

A team is automating deployment with GitHub Actions. An engineer proposes generating a personal access token from his own account, then storing it as a repository secret. The workflow would use it to deploy to production.

It works in a trial run. The team asks whether this is the right long-term arrangement.

What should they do instead?

- ☐ A Nothing; a PAT in a repository secret is the standard approach
- ☐ B Use OAuth M2M with a dedicated service principal, supplied to the workflow as environment variables
- ☐ C Use OAuth U2M so the workflow logs in interactively at deploy time
- ☐ D Commit the token to a config file in the repository so the workflow can read it

◆ PAUSE AND DECIDE

What should they do instead?

- A

A PAT in a repository secret

It works, and it ties production deployment to one person's account and uses the option Databricks now labels legacy.
- B

OAuth M2M with a service principal

Correct. No interactive sign-in, short-lived tokens, and production deployment belongs to a service principal rather than an individual.
- C

OAuth U2M in the workflow

U2M requires a browser sign-in. There is nobody there to complete it, which is the defining constraint of CI.
- D

Commit the token

Puts a live credential into Git history permanently, where it cannot be removed by a later commit.

THE SIGNAL

Two problems, not one: the authentication method is legacy, and the identity is a person. M2M with a service principal fixes both.

A bundle deploys correctly from an engineer's laptop with databricks bundle deploy -t dev. The same command in a CI job fails immediately, reporting that the CLI is not configured. The CI secret store holds a valid service principal client id and secret.

What is missing?

- ☐ A The bundle needs a profile name added to the target definition
- ☐ B The credentials are in the secret store but not set as environment variables for the step
- ☐ C The CI runner needs the Databricks CLI reinstalled
- ☐ D The service principal has not been granted CAN MANAGE on the bundle

◆ PAUSE AND DECIDE

What is missing?

A Add a profile name to the target

A profile name refers to an entry in .databrickscfg, and the runner has no such file. Naming a profile that does not exist changes nothing.

C Reinstall the CLI

The CLI ran and produced a configuration error, which means it is installed and executing.

B Credentials not exposed as environment variables

Correct. A fresh CI container has no .databrickscfg. Environment variables are how the credential reaches it, and they take precedence.

D Grant CAN MANAGE on the bundle

A permissions problem produces an authorisation error, not 'not configured'. The CLI never got as far as authenticating.

THE SIGNAL

'It works on my laptop' is the whole clue. The laptop has a .databrickscfg written by databricks auth login. The runner has nothing.

A team's pipeline deploys to production whenever main is updated. There is no validate step. A pull request renames a variable in databricks.yml but misses one reference elsewhere in the file.

The change is reviewed, approved and merged. The production deployment fails part way through, leaving some resources updated and others not.

What would have prevented this?

- ☐ A A more thorough code review
- ☐ B A databricks bundle validate step on the pull request, failing the check before merge
- ☐ C Deploying to production with --force
- ☐ D Running databricks bundle destroy before each deployment

◆ PAUSE AND DECIDE

What would have prevented this?

- A

A more thorough review

A dangling variable reference in a long YAML file is exactly what humans miss and machines catch instantly. Asking people to be more careful is not a control.
- B

A validate step on the pull request

Correct. Validate resolves the configuration without touching a workspace, so an unresolved reference fails the check before anyone merges.
- C

Deploy with --force

Overrides validations. It would make the pipeline more willing to deploy broken configuration, not less.
- D

Destroy before each deploy

Tears down production before deploying to it, converting a partial failure into a total outage.

THE SIGNAL

The objective names validate before deploy and manage. This scenario is what that ordering is for.

An urgent fix is needed in production. The pipeline's production deploy fails, reporting that the current Git branch does not match the branch specified in the target. The engineer is on hotfix/urgent-fix.

A colleague suggests permanently adding `--force` to the pipeline's production deploy step so this never blocks a release again.

What should the team do?

- ☐ A Add `--force` permanently, since urgent fixes should never be blocked
- ☐ B Merge the hotfix to main and let the pipeline deploy from there; keep the branch validation
- ☐ C Remove the git property from the production target
- ☐ D Change the production target's branch to hotfix/urgent-fix

◆ PAUSE AND DECIDE

What should the team do?

- A

 Add --force permanently
Turns a safety check into decoration. It still runs, still reports, and can no longer stop anything.
- B

 Merge to main and deploy from there
Correct. The hotfix goes through review and lands in main, so production matches the repository and the validation keeps working.
- C

 Remove the git property
Silently removes the check rather than overriding it. Same outcome as A, and harder to notice afterwards.
- D

 Point production at the hotfix branch
Production now tracks a temporary branch. When it is deleted, production's declared source no longer exists.

THE SIGNAL

Urgency is the argument every safety check eventually faces. The question is whether the check was wrong, and here it was right.

No human is present. **So the credential must be a service principal, the target must be explicit, and the gates must be in the pipeline.**